Дисциплина «Проектирование архитектуры ПО»

Кафедра корпоративных информационных систем

ОТЧЁТ

по практической работе №3

Тема: Проектирование логической и физической структуры данных

Выполнил: студент группы БСБО-01-23

Судовский Андрей Леонидович

Москва, 2026

СОДЕРЖАНИЕ

1. Цель работы 3

2. Задание 3

3. Логическая модель данных (ER-диаграмма) 4

4. Физическая схема PostgreSQL 6

5. Схема размещения данных по хранилищам 8

6. Структура данных в Redis 10

7. Модель графа Neo4j 11

8. Структура коллекций MongoDB 13

9. Маппинг Elasticsearch 15

10. Обоснование выбора хранилищ 17

11. Тестовые данные и запросы 18

12. Диаграмма потоков данных 19

13. Выводы 20

Список использованных источников 21

Приложение А. DDL-скрипт PostgreSQL (schema.sql) 23

Приложение Б. Маппинг индекса lectures (es_lectures_mapping.json) 26

1. Цель работы

Спроектировать логическую и физическую структуру данных системы управления учебным процессом университета с размещением данных по нескольким типам хранилищ (polyglot persistence), обосновать выбор хранилища для каждого участка данных, развернуть структуру в PostgreSQL, Redis, Neo4j, MongoDB и Elasticsearch и заполнить её тестовыми данными.

2. Задание

«На основе информации о группах учащихся, курсах обучения, лекционной программе и составу лекционных курсов и практических занятий, а также структуре связей между курсами, специальностями, студентами кафедры и данными о посещении студентами занятий, сформировать структуру хранения и связей в реляционной базе данных. Разместить информацию по различным видам представления хранения данных (структуры ключ-значение, объекты документ-композиция, наборы типизированных связей, полнотекстовая информация с метаданными, транзакционные данные с партицированием) для обеспечения оптимальной структуры для выборки информации в целях аналитических запросов. Обосновать свой выбор с точки зрения характеристик типов хранилищ».

Отчёт построен по требованиям методических указаний: ER-диаграмма (раздел 3), физическая схема PostgreSQL (4), схема размещения данных (5), структуры Redis (6), Neo4j (7), MongoDB (8) и Elasticsearch (9), обоснование выбора (10), тестовые данные с запросами (11), диаграмма потоков данных (12).

За основу взята модель методических указаний. Она дополнена сведениями, которые нужны аналитическим запросам лабораторных работ №1–3 и учёту курсов по выбору из практической работы №2: у занятия — требования к техническим средствам и теги, у курса — кафедра-владелец, признак курса по выбору и теги, выбор курсов студентами — отдельной связью. Все расширения перечислены в таблице 2.

Работа выполнена по методическим указаниям [2] к заданию [1]. Физическая схема и секционирование — по документации PostgreSQL [3, 4], структуры дублей — по документации Redis, MongoDB, Neo4j и Elasticsearch [5–8], захват изменений — по документации Debezium [9]; принципы polyglot persistence и выбора модели данных — по [10, 11].

3. Логическая модель данных (ER-диаграмма)

Предметная область — учебный процесс университета: организационная структура (университет, институт, кафедра, специальность), учебный план (курс, занятие, материалы занятия), контингент (группа, студент) и учёт занятий (расписание, журнал посещений). Логическая модель приведена на рисунке 1 в нотации «вороньих лапок»: «||» — ровно один, «o|» — ноль или один, «}|» — один или много, «}o» — ноль или много.

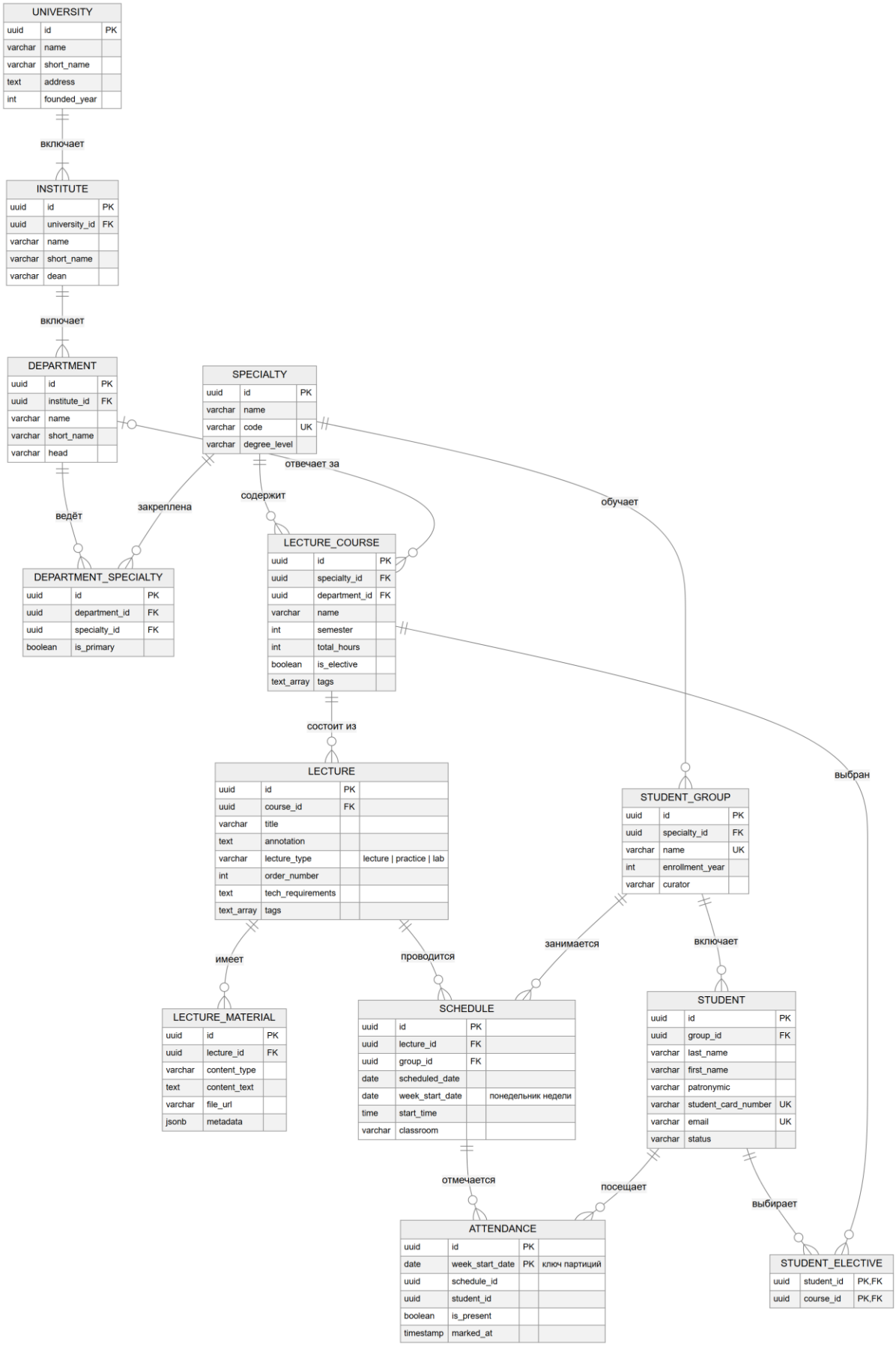


Рисунок 1 — ER-диаграмма предметной области (логическая модель)

Таблица 1 — Связи логической модели

Связь	**Кардинальность**	**Смысл**
Университет → институт → кафедра	1:N, 1:N	организационная иерархия
Кафедра ↔ специальность	M:N (department_specialties)	специальность могут вести несколько кафедр, одна из них выпускающая (`is_primary`)
Специальность → курс, специальность → группа	1:N, 1:N	учебный план и группы направления
Кафедра → курс	1:N (необязательная)	кафедра, отвечающая за курс (тег «спецдисциплина кафедры»)
Курс → занятие → материал	1:N, 1:N	занятия трёх типов: лекция, практика, лабораторная
Группа → студент	1:N	состав группы
Студент ↔ курс	M:N (student_electives)	выбор курсов по выбору
Занятие + группа → расписание	M:N (schedule)	конкретное занятие группы на дату и время
Расписание + студент → посещение	M:N (attendance)	отметка о посещении

4. Физическая схема PostgreSQL

PostgreSQL — основное хранилище и источник истины: в нём все 13 таблиц и все связи с ограничениями целостности (база `university_system`). DDL-скрипт с комментариями — `pr3\schema.sql`, полный текст — приложение А. Основные решения:

– первичные ключи — UUID: один и тот же идентификатор используется во всех пяти хранилищах, не нужен центральный счётчик и не возникает конфликтов при синхронизации;

– поля `created_at` / `updated_at` в каждой таблице — для отслеживания изменений и захвата изменений (CDC) в лабораторных работах №4–6;

– внешние ключи с `ON DELETE CASCADE` поддерживают целостность иерархии; уникальные ограничения — код специальности, название группы, номер зачётной книжки, e-mail, пара «курс — номер занятия»;

– индексы B-tree по внешним ключам и частым фильтрам (дата, неделя, группа + дата), GIN по массивам тегов и триграммный GIN по ФИО (`pg_trgm`) для нечёткого поиска;

– журнал посещений `attendance` секционирован `PARTITION BY RANGE (week_start_date)`: 20 недельных партиций весеннего семестра 2026 года и партиция по умолчанию; первичный ключ (`id`, `week_start_date`) включает ключ партиционирования, внешние ключи у журнала не объявлены, как в методических указаниях (быстрая массовая вставка, свободное отсоединение партиций);

– `schedule.week_start_date` денормализован (понедельник недели) и проверяется ограничением `CHECK (week_start_date = date_trunc('week', scheduled_date))`;

– представления `v_full_schedule` (расписание со всей иерархией), `v_student_attendance_stats` (посещаемость студента) и `v_course_listeners` (слушатели курса с учётом курсов по выбору).

Таблица 2 — Расширения схемы

Таблица	**Расширение относительно методических указаний**	**Для чего**
lecture_courses	`department_id`, `is_elective`, `tags TEXT[]`	кафедра-владелец, курсы по выбору, тег «спецдисциплина кафедры» (ЛР №3)
lectures	`tech_requirements TEXT`, `tags TEXT[]`	требования к технике (ЛР №2), темы и теги занятия (ЛР №1, №3)
student_electives	новая таблица (студент, курс)	курсы по выбору (ПР №2), число слушателей (ЛР №2)
students	триграммный индекс по ФИО	поиск студента по фамилии с опечатками
schedule	ограничение `CHECK` на `week_start_date`	ключ партиционирования журнала всегда верен
—	представление `v_course_listeners`	кто слушает курс: вся группа или записавшиеся

Развёрнутая схема — 13 таблиц, 21 партиция журнала посещений и 3 представления (рисунок 2).

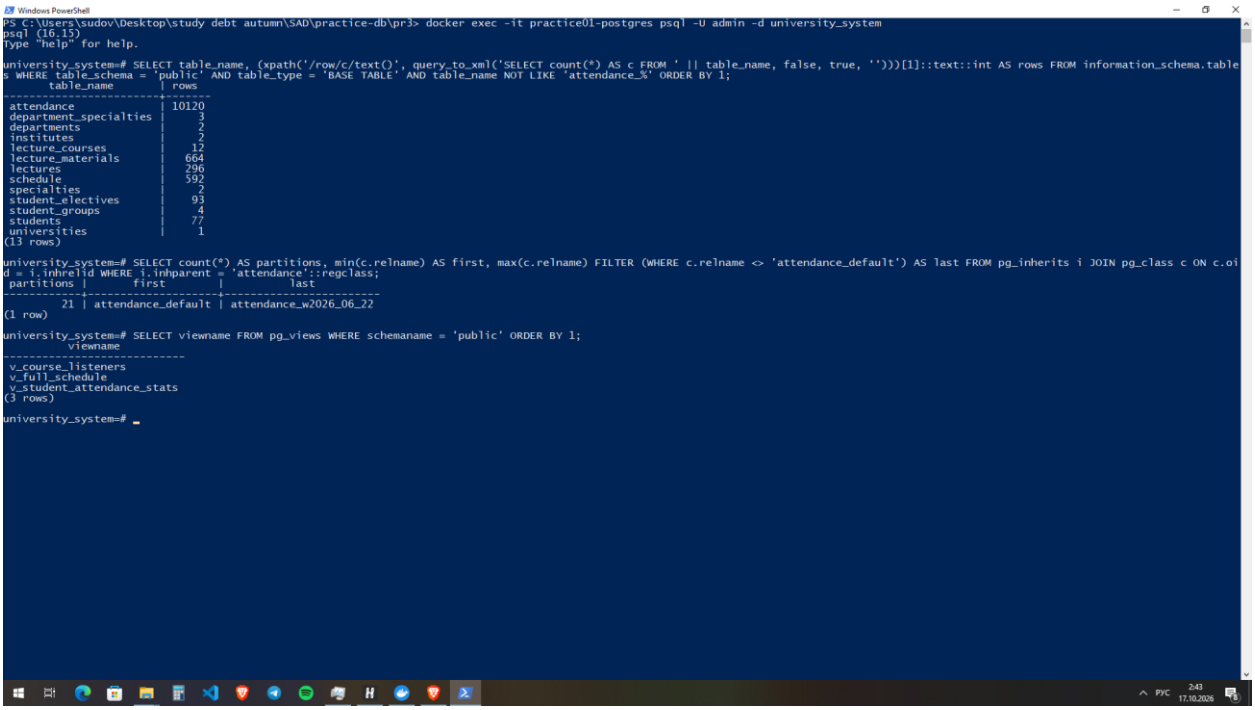


Рисунок 2 — PostgreSQL: таблицы и число строк, партиции журнала посещений, представления

5. Схема размещения данных по хранилищам

PostgreSQL хранит все данные; остальные хранилища содержат дубли отдельных участков, представленные в модели, которая лучше подходит для задачи чтения (рисунок 3, таблица 3). Каждой форме из задания соответствует своё хранилище: ключ-значение — Redis, документ-композиция — MongoDB, наборы типизированных связей — Neo4j, полнотекстовая информация с метаданными — Elasticsearch, транзакционные данные с партиционированием — PostgreSQL.

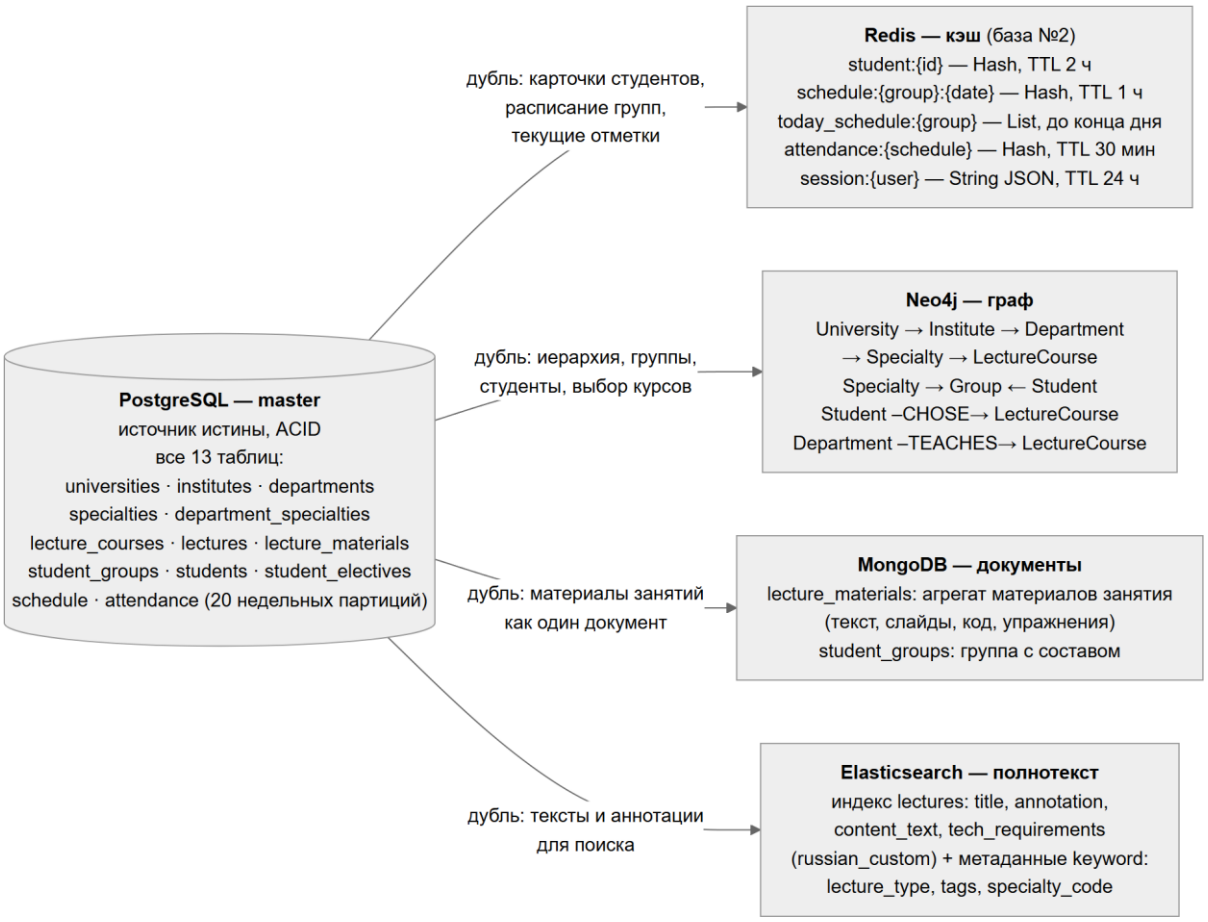


Рисунок 3 — Схема размещения данных по хранилищам

Таблица 3 — Карта данных по хранилищам

Участок данных	**PostgreSQL**	**Redis**	**Neo4j**	**MongoDB**	**Elasticsearch**
Университеты, институты, кафедры	master	—	дубль (граф)	—	—
Специальности, курсы	master	—	дубль (граф)	—	название курса в индексе
Занятия: заголовок, аннотация, техника, теги	master	—	—	в документе материалов	дубль (поиск)
Тексты и материалы занятий	master (строки)	—	—	дубль (документ)	дубль (полнотекст)
Группы и студенты	master	кэш карточек	дубль (граф)	группа с составом	—
Выбор курсов по выбору	master	—	дубль (связи)	в составе группы	—
Расписание	master	дубль (кэш)	—	—	—
Журнал посещений	master (партиции)	буфер открытого занятия	—	—	—
Сессии пользователей	—	только здесь	—	—	—

6. Структура данных в Redis

Redis используется как кэш (логическая база №2) по шаблону cache-aside: приложение сначала читает ключ из Redis, при промахе читает PostgreSQL и кладёт результат в Redis с временем жизни (TTL). Redis не является источником истины: потеря кэша означает только временное замедление чтения.

Таблица 4 — Ключи Redis

Ключ	**Тип**	**TTL**	**Источник в PostgreSQL**	**Назначение**
`student:{student_id}`	Hash	2 ч	students	карточка студента для быстрого доступа
`schedule:{group_id}:{date}`	Hash: slot_N → JSON	1 ч	schedule + lectures + lecture_courses	расписание группы на день, читают все студенты группы
`today_schedule:{group_id}`	List	до конца дня (`EXPIREAT`)	schedule	занятия группы на сегодня по порядку
`attendance:{schedule_id}`	Hash: зачётка → present/absent	30 мин	attendance	отметки открытого занятия до сохранения в журнал
`session:{user_id}`	String (JSON)	24 ч	—	сессия пользователя (только в Redis)

Примеры команд: `HGETALL schedule:{group_id}:2026-03-16` — расписание группы на день, `TTL` — оставшееся время жизни, `LRANGE today_schedule:{group_id} 0 -1` — список занятий, `GET session:student-23b0101` — сессия. Загрузка выполняется через `redis-cli --pipe` (60 команд: 31 ключ вместе с `EXPIRE`). Результаты — рисунок 4, вид в Redis Commander со значением TTL — рисунок 5.

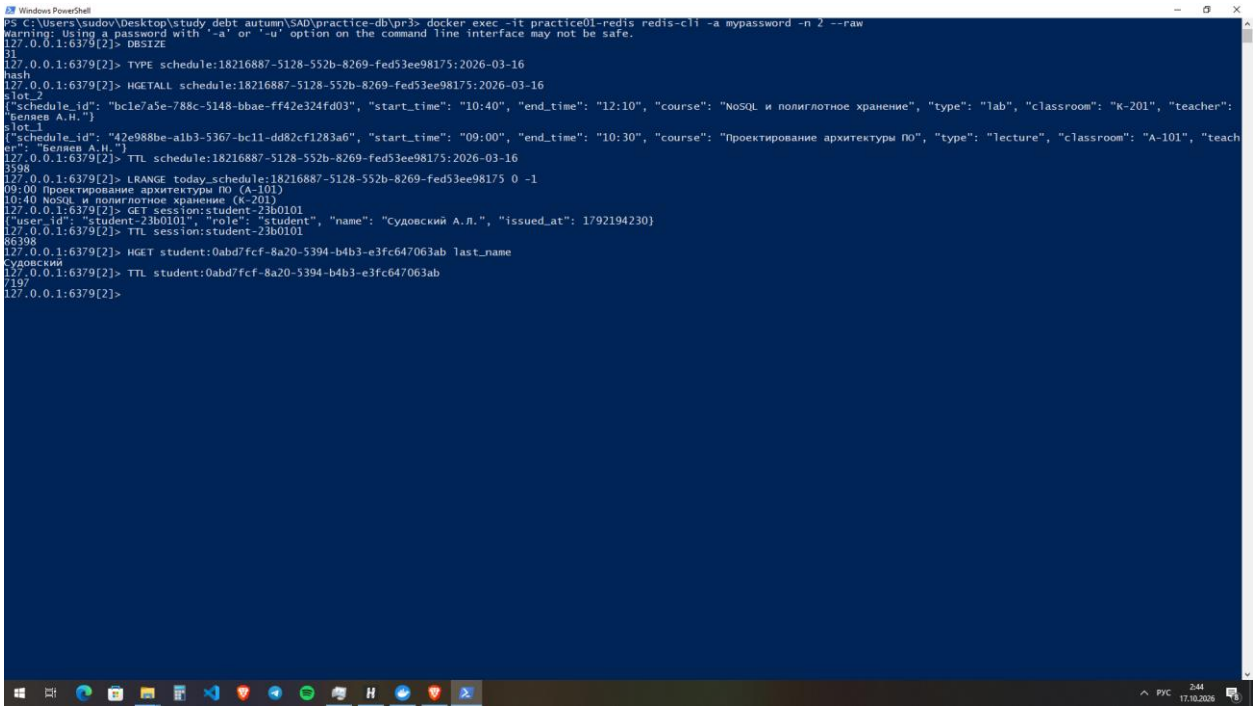


Рисунок 4 — Redis: структура кэша, значения и время жизни ключей

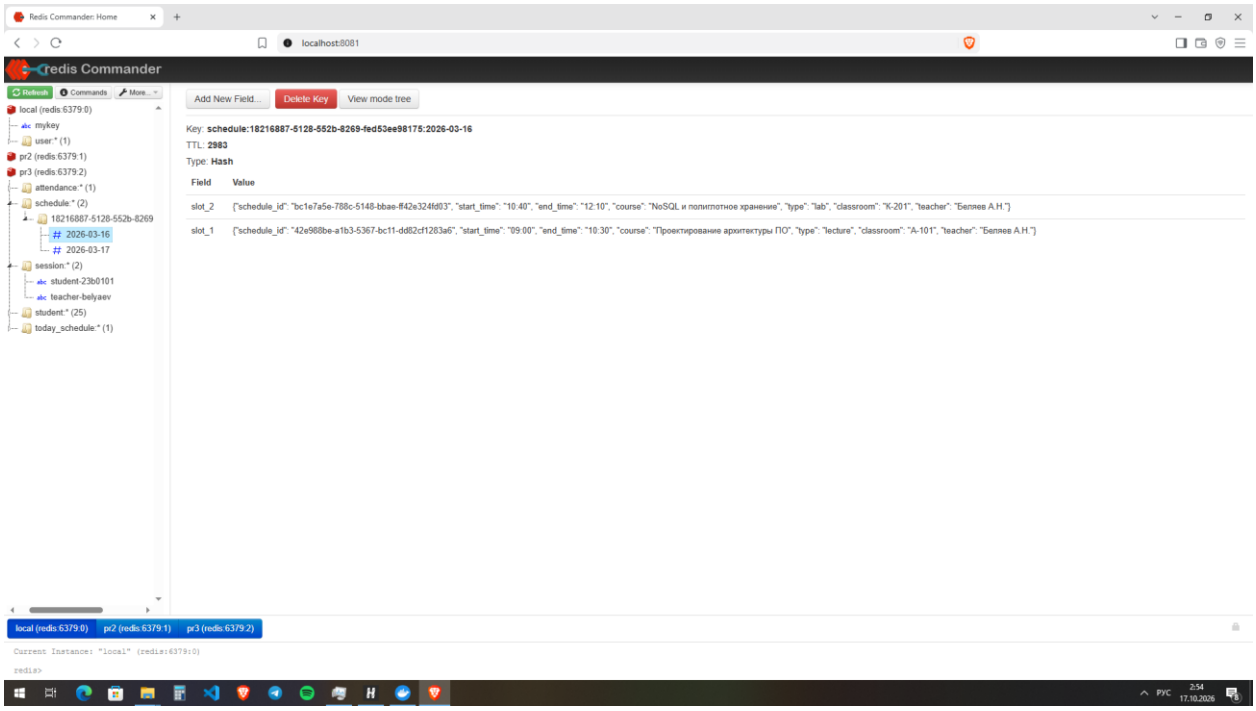


Рисунок 5 — Redis Commander: хеш расписания группы на день и его TTL

7. Модель графа Neo4j

Neo4j хранит дубль иерархии «университет → институт → кафедра → специальность → курс» и связи контингента с курсами. Свойства узлов — `id` (тот же UUID, что в PostgreSQL, с ограничением уникальности) и поля для отображения и фильтрации.

Таблица 5 — Узлы графа

Метка	**Свойства**	**Узлов**
University	id, name, short_name	1
Institute	id, name, short_name, dean	2
Department	id, name, short_name, head	2
Specialty	id, name, code, degree_level	2
LectureCourse	id, name, semester, total_hours, is_elective, tags	12
Group	id, name, enrollment_year	4
Student	id, student_card_number, name	77

Таблица 6 — Связи графа

Связь	**Смысл**	**Связей**
(University)-[:HAS_INSTITUTE]->(Institute)	университет включает институт	2
(Institute)-[:HAS_DEPARTMENT]->(Department)	институт включает кафедру	2
(Department)-[:HAS_SPECIALTY {is_primary}]->(Specialty)	кафедра ведёт специальность (M:N)	3
(Specialty)-[:HAS_COURSE]->(LectureCourse)	курс учебного плана специальности	12
(Department)-[:TEACHES]->(LectureCourse)	кафедра отвечает за курс	12
(Specialty)-[:HAS_GROUP]->(Group)	группа обучается по специальности	4
(Student)-[:MEMBER_OF]->(Group)	студент в группе	77
(Student)-[:CHOSE]->(LectureCourse)	выбор курса по выбору	93

Иерархия и группы показаны на рисунке 6. Примеры запросов (рисунок 7): полный путь от университета до курса; «хлебные крошки» через `shortestPath` (РТУ МИРЭА → ИКБ → КИС → NoSQL и полиглотное хранение); все курсы института за два перехода; число слушателей курса — студенты групп специальности для обязательного курса и только выбравшие (`CHOSE`) для курса по выбору. Последний запрос — основа ЛР №2.

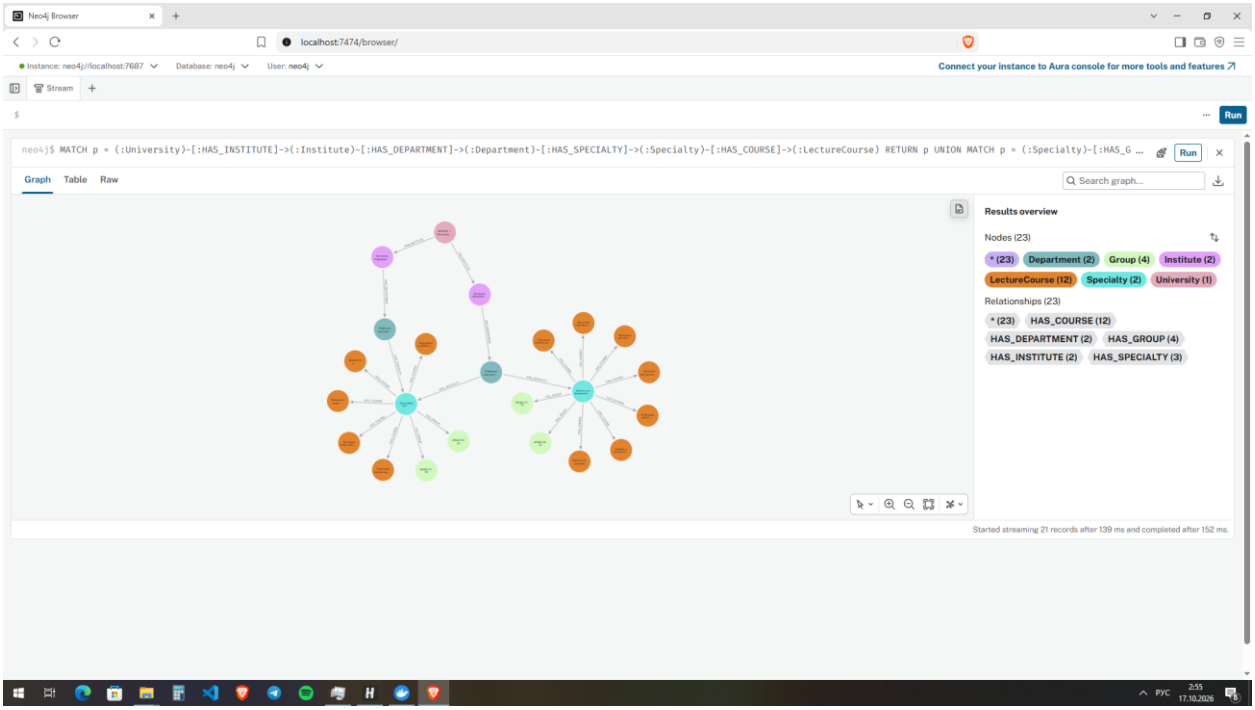


Рисунок 6 — Neo4j Browser: иерархия университета, курсы и группы

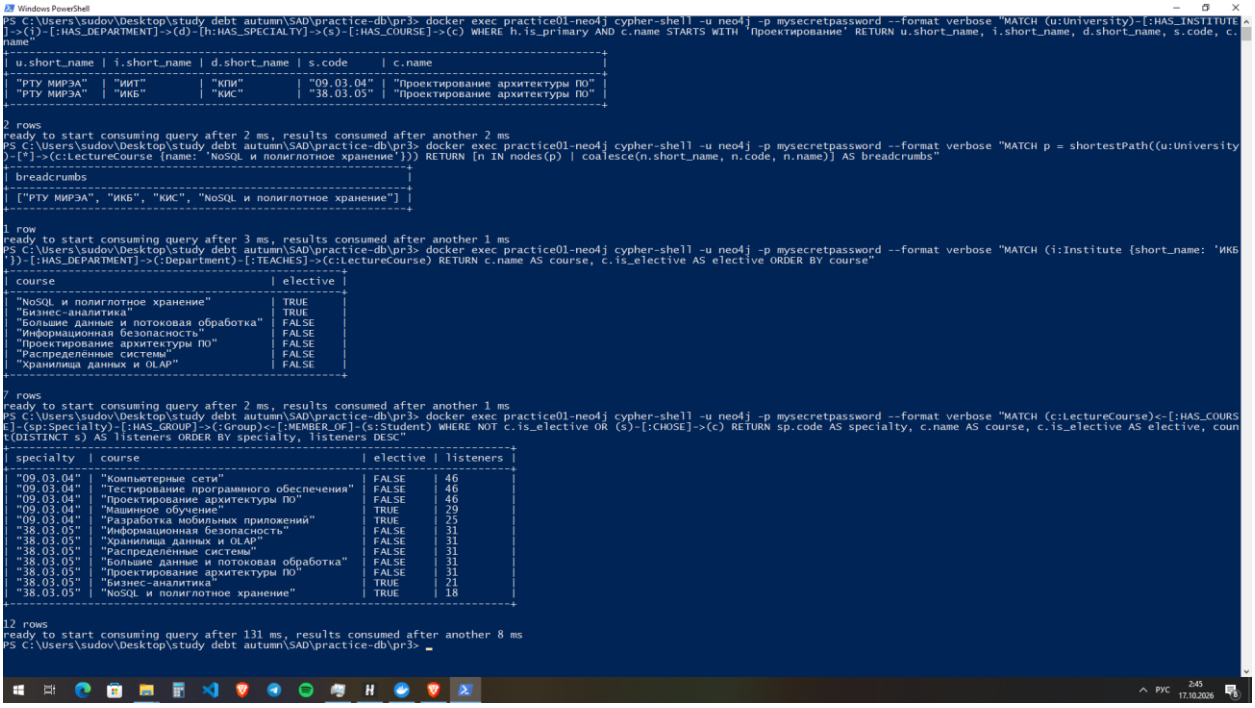


Рисунок 7 — Neo4j: запросы обхода иерархии и подсчёта слушателей курсов

8. Структура коллекций MongoDB

Коллекция `lecture_materials` — один документ на занятие: агрегат, содержащий всё для страницы занятия. `_id` совпадает с UUID занятия. Структура документа:

– `lecture_id`, `course_id`, `course_name`, `lecture_title`, `lecture_type`, `order_number` — денормализованные сведения о занятии и курсе, чтобы не обращаться к PostgreSQL;

– `content` — основной текст: формат, текст, число слов, время чтения;

– `materials` — массив материалов разной структуры: слайды (`file_url`, `pages`), пример кода (`language`, `code`), упражнение (`questions` с уровнем сложности), ссылка на видео — у каждого типа свой набор полей, что в реляционной модели требует либо разреженной таблицы, либо JSONB;

– `metadata` — автор, дата обновления (`Date`), версия, теги, кафедра, уровень сложности.

Коллекция `student_groups` — группа с вложенным составом студентов и их курсами по выбору (документ-композиция из практической работы №2). Индексы: `{course_id, order_number}`, `{metadata.tags}`, `{materials.type}` для `lecture_materials` и уникальный `{name}` для `student_groups`. Документ — рисунок 8, запросы (материалы занятия одним запросом, поиск по типу вложенного материала, агрегация по курсам) — рисунок 9.

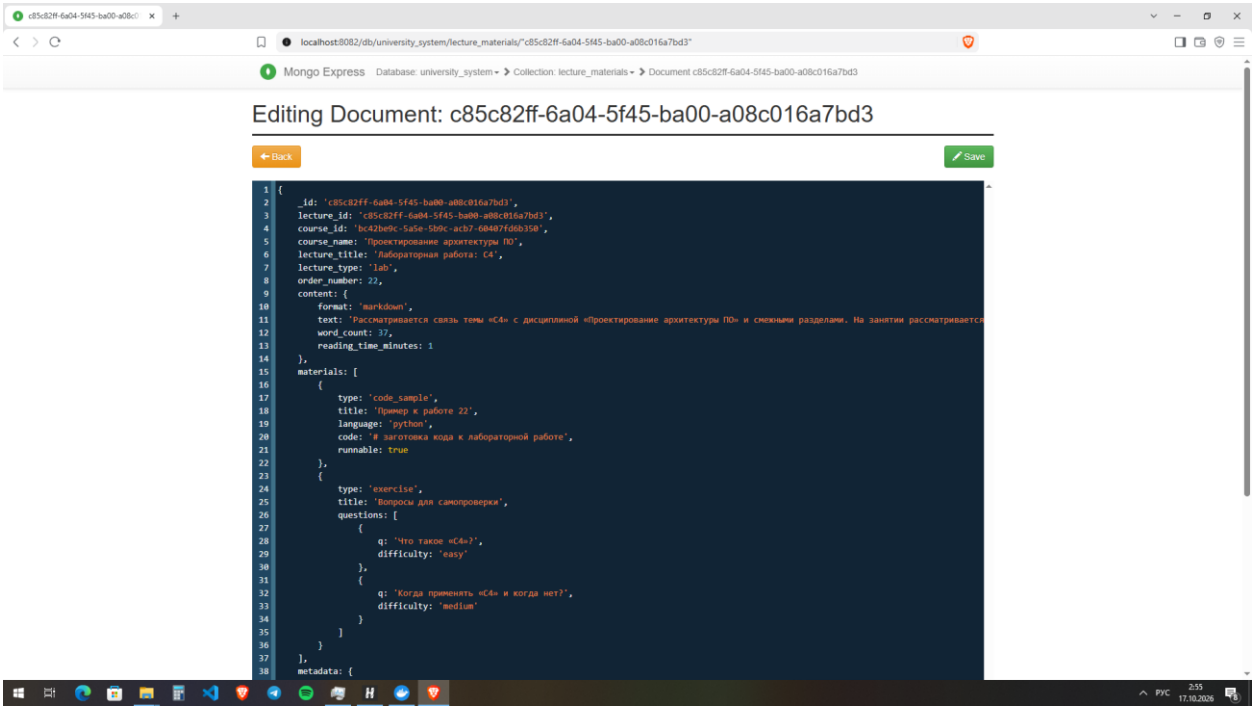


Рисунок 8 — Mongo Express: документ коллекции lecture_materials

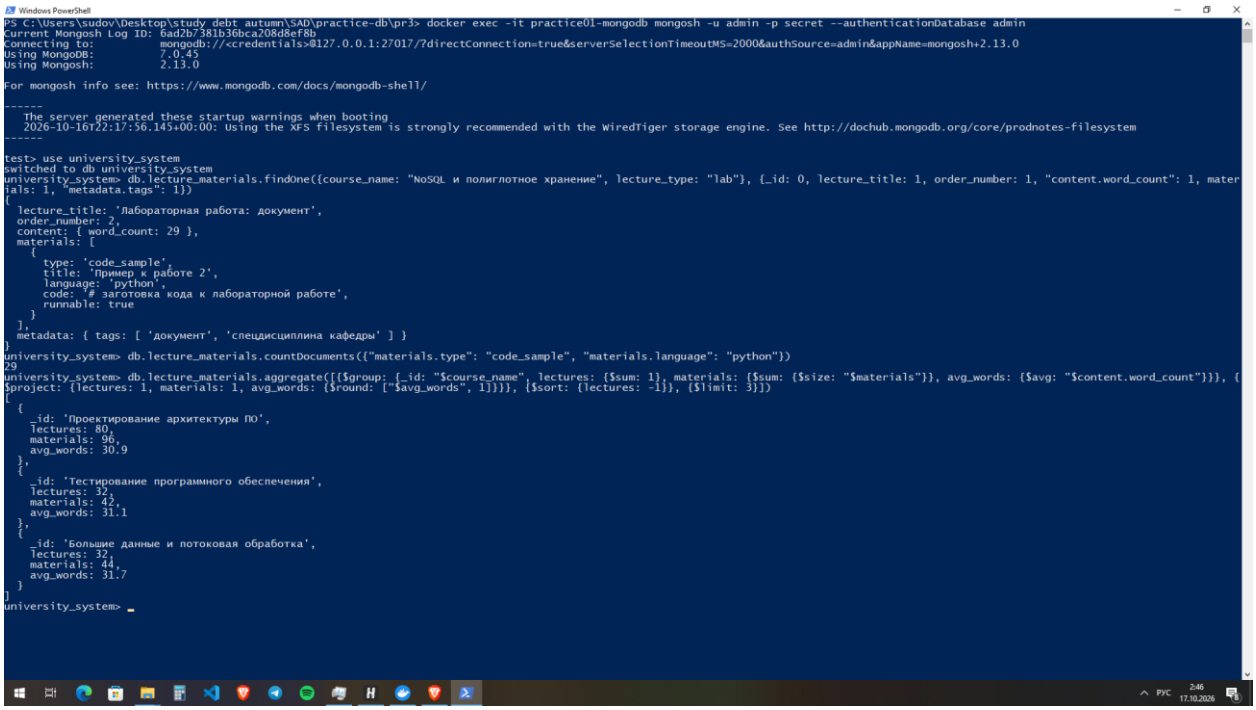


Рисунок 9 — MongoDB: чтение агрегата занятия, поиск по вложенным материалам, агрегация

9. Маппинг Elasticsearch

Индекс `lectures` (один шард, без реплик — стенд с одним узлом) хранит тексты занятий для полнотекстового поиска. Анализатор `russian_custom`: токенизатор `standard`, фильтры `lowercase`, `russian_stop` (стоп-слова «и», «в», «на»…) и `russian_stemmer` (приведение к основе: «партиционированием» → «партиционирован»). Поля:

Таблица 7 — Маппинг индекса lectures

Поле	**Тип**	**Назначение**
title, course_name	`text` (russian_custom) + подполе `keyword`	поиск по словам и точная фильтрация, сортировка
annotation, content_text	`text` (russian_custom)	полнотекстовый поиск по аннотации и тексту
tech_requirements	`text` (russian_custom)	поиск требований к технике (ЛР №2)
lecture_type, tags, specialty_code, department	`keyword`	фильтры и агрегации (ЛР №3 — тег спецдисциплины)
lecture_id, course_id	`keyword`	связь с PostgreSQL по UUID
order_number, semester	`integer`	сортировка, диапазоны

Примеры запросов (рисунки 10–12, тела запросов — каталог `pr3\queries`): `multi_match` по трём полям с весами (`title^3`, `annotation^2`) и `fuzziness: AUTO` с подсветкой; `bool`-запрос

«лабораторные спецдисциплин кафедры, где требуется Docker» (`must` по тексту требований и `filter` по `keyword`-полям); агрегация числа занятий по типам и направлениям. Полный маппинг — приложение Б.

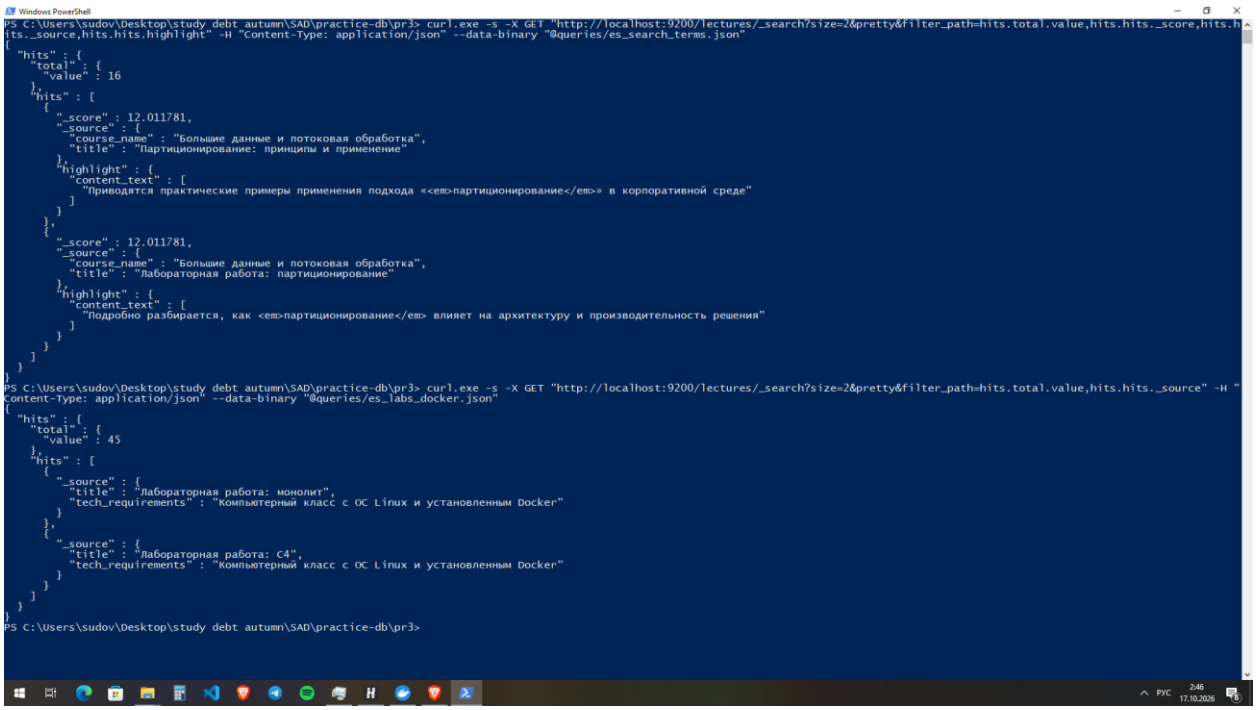


Рисунок 10 — Elasticsearch: полнотекстовый поиск с подсветкой и комбинированный запрос bool

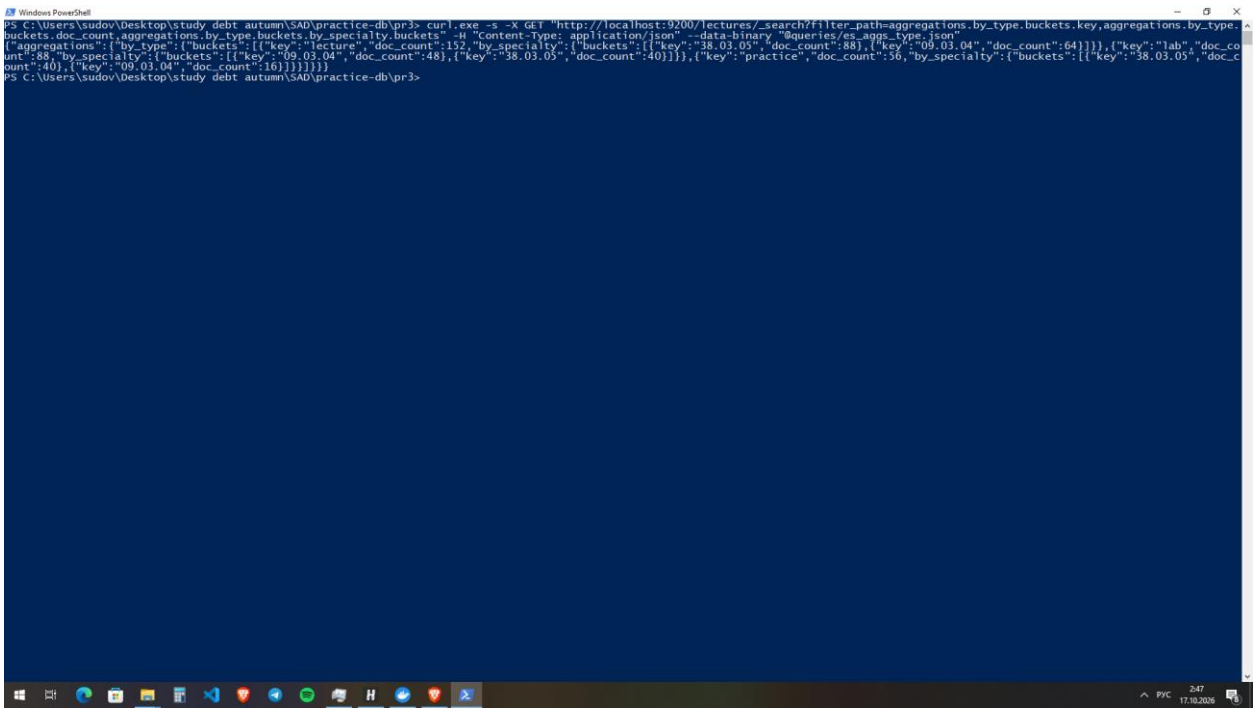


Рисунок 11 — Elasticsearch: агрегация занятий по типам и направлениям

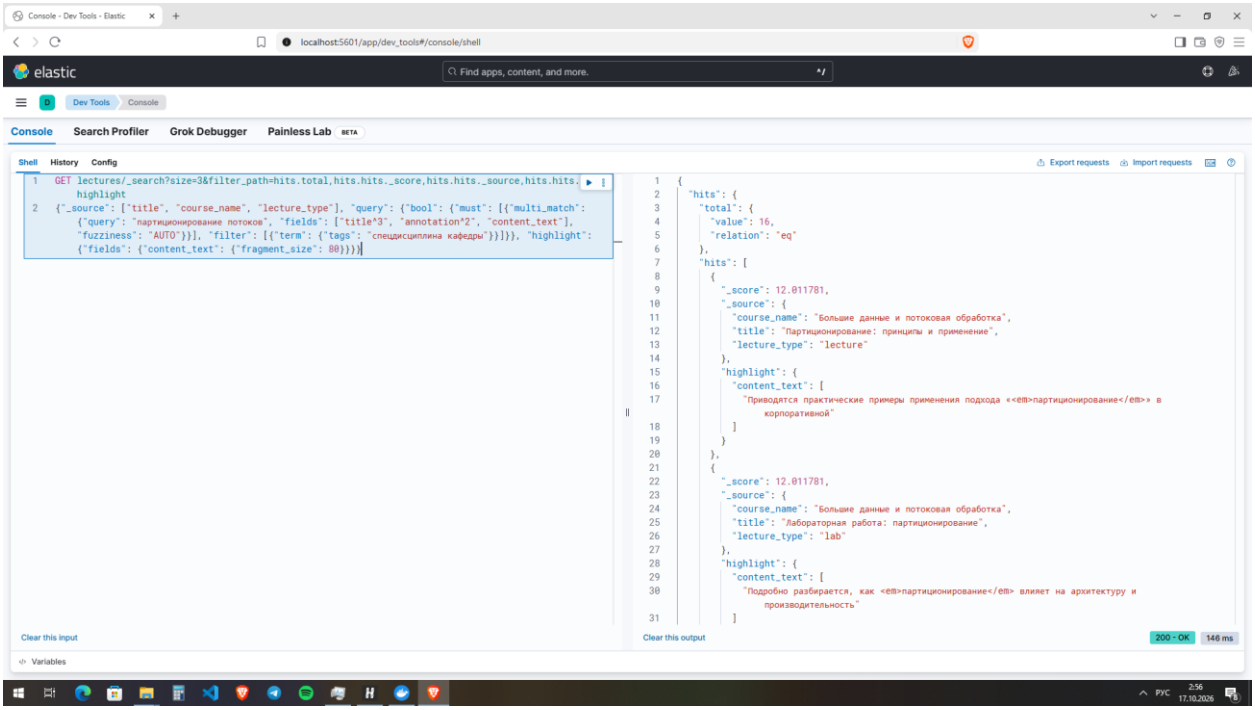


Рисунок 12 — Kibana Dev Tools: поиск по текстам занятий с фильтром по тегу и подсветкой

10. Обоснование выбора хранилищ

Таблица 8 — Обоснование выбора хранилищ

Участок данных	Хранилище	Почему именно оно
Все сущности и связи, редактирование	PostgreSQL	ACID-транзакции, внешние ключи и ограничения, сложные JOIN и агрегаты — единственный источник истины
Журнал посещений	PostgreSQL, секционирование по неделям	самая быстрорастущая таблица; запрос за неделю читает одну партицию (partition pruning), старые недели отсоединяются и архивируются целиком
Расписание, карточки студентов, отметки открытого занятия, сессии	Redis	чтение по ключу за O(1) из памяти, автоматическое устаревание по TTL; данные читаются намного чаще, чем меняются
Иерархия и связи «группа — студент — курс»	Neo4j	обход связей без JOIN: путь «университет → … → курс» — один шаблон вместо четырёх соединений, `shortestPath` для навигации, подсчёт слушателей с учётом выборов
Материалы занятий	MongoDB	разнородная структура материалов (слайды, код, упражнения, видео) естественна для документа; страница занятия — одно чтение документа
Тексты и аннотации, требования к технике	Elasticsearch	морфологический анализ русского языка, ранжирование BM25, нечёткий поиск, подсветка, агрегации по метаданным

Дублирование оправдано тем, что каждое хранилище отвечает за свой тип запросов, а запись по-прежнему идёт в одно место — PostgreSQL. Согласованность дублей обеспечивает захват изменений (CDC) из журнала

PostgreSQL (лабораторные работы №4–6); для кэша Redis дополнительно действует TTL: даже без синхронизации устаревшие данные живут не дольше заданного срока.

11. Тестовые данные и запросы

Тестовый набор формирует детерминированный генератор (`common\dataset3.py`, на основе набора ПР №2: те же группы БСБО и их студенты), файлы для хранилищ — `pr3\make_data.py`, загрузку во все пять хранилищ — `pr3\load_all.sh`. Отметки посещаемости формируются случайным выбором в SQL (`random()` с индивидуальной вероятностью студента, `setseed` для воспроизводимости).

Таблица 9 — Объём тестовых данных

Хранилище	**Объём данных**
PostgreSQL	1 университет, 2 института, 2 кафедры, 2 специальности, 3 связи кафедра — специальность, 12 курсов, 296 занятий (152 лекции, 56 практик, 88 лабораторных), 664 материала, 4 группы, 77 студентов, 93 выбора курсов, 592 занятия в расписании, 10 120 отметок посещаемости в 16 недельных партициях (посещаемость 75,1 %)
Redis	31 ключ: 25 карточек студентов, 2 расписания на день, список занятий на сегодня, буфер отметок, 2 сессии
Neo4j	100 узлов и 205 связей
MongoDB	296 документов lecture_materials, 4 документа student_groups
Elasticsearch	296 документов индекса lectures

Запросы к Redis, Neo4j, MongoDB и Elasticsearch приведены в разделах 6–9. Запросы к PostgreSQL (рисунок 13): расписание группы на день через `v_full_schedule`, студенты с наименьшей посещаемостью через `v_student_attendance_stats`, посещаемость группы за неделю — план показывает чтение одной партиции `attendance_w2026_03_16`, и закрепление специальностей за кафедрами (M:N).

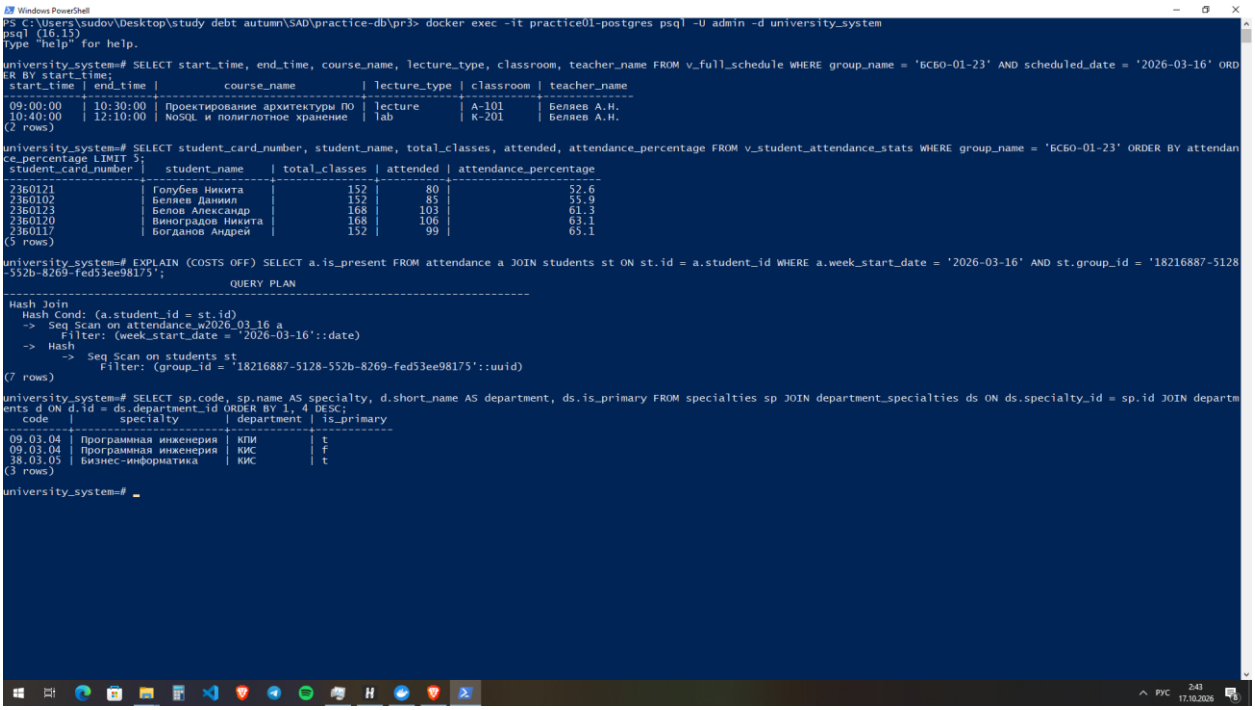
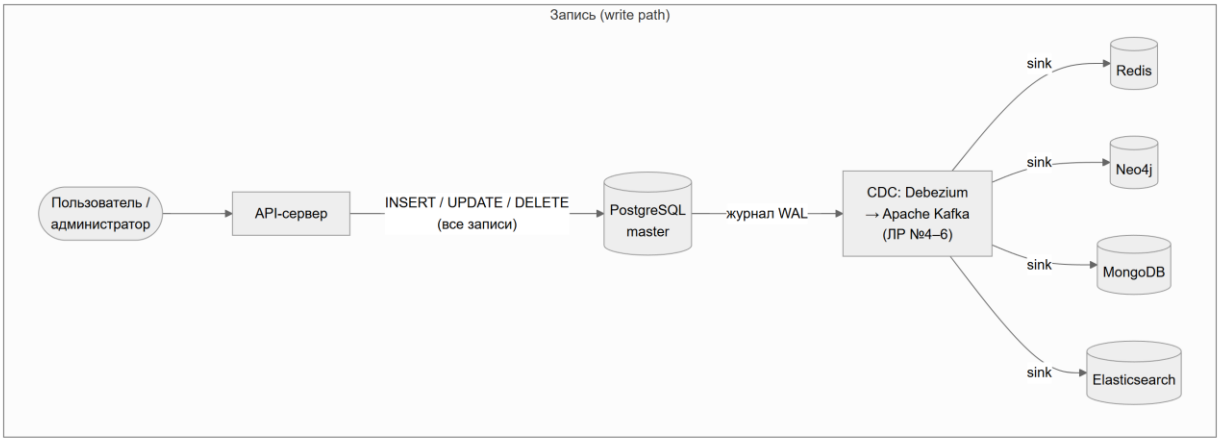


Рисунок 13 — PostgreSQL: запросы к представлениям, отсечение партиций, связь M:N

12. Диаграмма потоков данных

Запись: все изменения (INSERT, UPDATE, DELETE) выполняются только в PostgreSQL. Изменения попадают в журнал WAL, Debezium передаёт их в Apache Kafka, а коннекторы-приёмники обновляют дубли в Redis, Neo4j, MongoDB и Elasticsearch — этот контур реализуется в лабораторных работах №4–6. Чтение: каждый сценарий обслуживается хранилищем, модель которого лучше подходит к запросу (рисунок 14).



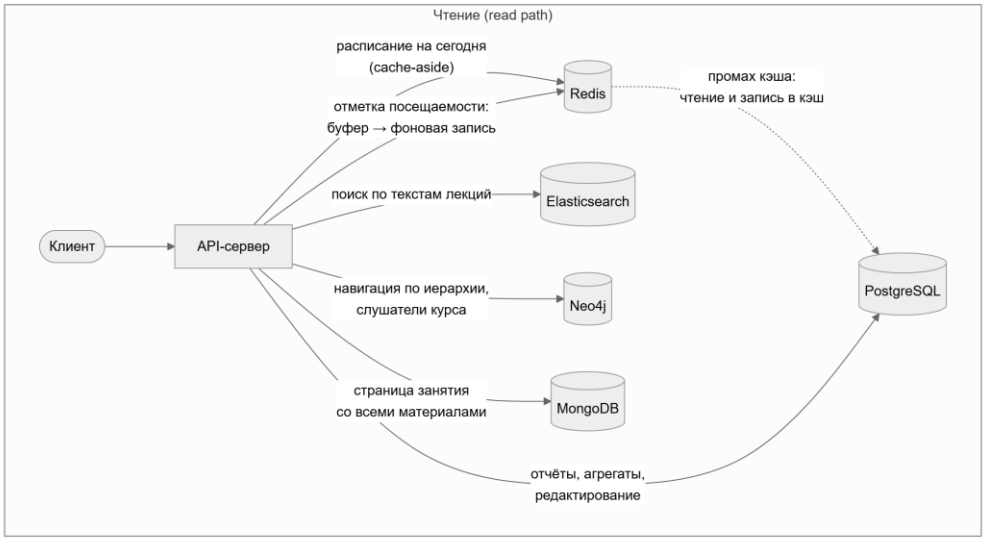


Рисунок 14 — Потоки данных: запись через PostgreSQL и CDC, чтение из хранилищ по сценариям

Таблица 10 — Сценарии чтения

Сценарий чтения	**Хранилище**	**Обоснование**
Расписание группы на сегодня	Redis → PostgreSQL при промахе	частые чтения, данные меняются редко
Поиск по текстам занятий	Elasticsearch	морфология, ранжирование, подсветка
Навигация по иерархии, слушатели курса	Neo4j	обход графа без соединений
Страница занятия с материалами	MongoDB	один документ — все материалы
Отметка посещаемости	Redis (буфер) → PostgreSQL	быстрая запись, затем фоновая фиксация
Отчёт по посещаемости за семестр	PostgreSQL	точные агрегаты и соединения, партиции

13. Выводы

Спроектирована структура данных системы учебного процесса: логическая модель из 13 сущностей с кардинальностями связей и физическая

схема PostgreSQL с ограничениями целостности, индексами, представлениями и журналом посещений, секционированным по неделям. Модель методических указаний дополнена тем, что требуется аналитическим запросам лабораторных работ: требованиями к технике и тегами занятий, кафедрой-владельцем курса и учётом курсов по выбору.

Данные размещены по принципу polyglot persistence: PostgreSQL — источник истины, Redis — кэш с ограниченным временем жизни, Neo4j — иерархия и связи «группа — студент — курс», MongoDB — материалы занятий как документы, Elasticsearch — полнотекстовый поиск с морфологией. Структура развёрнута во всех пяти хранилищах, заполнена согласованными данными с общими UUID и проверена запросами к каждому из них. Синхронизация дублей с основным хранилищем закладывается через CDC и будет реализована в лабораторных работах №4–6.

Работа выполнена 17.10.2026; рисунки сняты в тот же день с экрана рабочего компьютера, схемы построены по описаниям в каталоге `pr3\diagrams`.

Список использованных источников

1. Задание на практические и лабораторные работы по дисциплине «Проектирование архитектуры ПО». — Москва, 2026. — Текст : электронный.

2. Практическая работа №3. Проектирование логической и физической структуры данных : методические указания. — 2026. — 34 с. — Текст : электронный.

3. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

4. PostgreSQL 16 Documentation. Table Partitioning. — URL: https://www.postgresql.org/docs/16/ddl-partitioning.html (дата обращения: 17.10.2026). — Текст : электронный.

5. Redis Documentation. — URL: https://redis.io/docs/latest/ (дата обращения: 17.10.2026). — Текст : электронный.

6. MongoDB Manual. — URL: https://www.mongodb.com/docs/manual/ (дата обращения: 17.10.2026). — Текст : электронный.

7. Neo4j Cypher Manual. — URL: https://neo4j.com/docs/cypher-manual/current/ (дата обращения: 17.10.2026). — Текст : электронный.

8. Elasticsearch Guide 8.16. — URL: https://www.elastic.co/guide/en/elasticsearch/reference/8.16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

9. Debezium Documentation. — URL: https://debezium.io/documentation/ (дата обращения: 17.10.2026). — Текст : электронный.

10. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Санкт-Петербург : Питер, 2018. — 640 с. — Текст : непосредственный.

11. Садаладж П. Дж., Фаулер М. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — Москва : Вильямс, 2013. — 192 с. — Текст : непосредственный.

Приложение А. DDL-скрипт PostgreSQL (schema.sql)

```
-- ================================================================================
-- ПР №3. Физическая модель PostgreSQL — основное хранилище (источник истины) системы
-- управления учебным процессом. База: university_system.
-- Основа — схема методических указаний; расширения помечены «РАСШИРЕНИЕ» и нужны
-- аналитическим запросам лабораторных работ №1–3 (термины, техника, теги, курсы по выбору).
-- ================================================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";   -- UUID как первичные ключи: один id во всех хранилищах
CREATE EXTENSION IF NOT EXISTS "pg_trgm";     -- нечёткий поиск по ФИО (триграммы)


-- ---------- Организационная структура ------------------------------------------
CREATE TABLE universities (
    id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name         VARCHAR(500) NOT NULL,           -- «МИРЭА — Российский технологический университет»
    short_name   VARCHAR(100),                    -- «РТУ МИРЭА»
    address      TEXT,
    website      VARCHAR(255),
    founded_year INT,
    created_at   TIMESTAMP DEFAULT NOW(),         -- created_at / updated_at нужны CDC (ЛР №4-6)
    updated_at   TIMESTAMP DEFAULT NOW()
);


CREATE TABLE institutes (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    university_id UUID NOT NULL REFERENCES universities(id) ON DELETE CASCADE,
    name          VARCHAR(500) NOT NULL,
    short_name    VARCHAR(100),                   -- «ИКБ», «ИИТ»
    dean          VARCHAR(300),                   -- директор института
    created_at    TIMESTAMP DEFAULT NOW(),
    updated_at    TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_institutes_university ON institutes(university_id);


CREATE TABLE departments (
    id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    institute_id UUID NOT NULL REFERENCES institutes(id) ON DELETE CASCADE,
    name         VARCHAR(500) NOT NULL,
    short_name   VARCHAR(100),                    -- «КИС», «КПИ»
    head         VARCHAR(300),                    -- заведующий кафедрой
    room         VARCHAR(50),
    created_at   TIMESTAMP DEFAULT NOW(),
    updated_at   TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_departments_institute ON departments(institute_id);


CREATE TABLE specialties (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name           VARCHAR(500) NOT NULL,
    code           VARCHAR(20) NOT NULL,          -- «38.03.05»
    degree_level   VARCHAR(20) NOT NULL CHECK (degree_level IN ('bachelor', 'master', 'specialist')),
    duration_years INT NOT NULL DEFAULT 4,
    created_at     TIMESTAMP DEFAULT NOW(),
    updated_at     TIMESTAMP DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_specialties_code ON specialties(code);


-- Кафедра <-> специальность (M:N): специальность может вестись несколькими кафедрами
CREATE TABLE department_specialties (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    department_id UUID NOT NULL REFERENCES departments(id) ON DELETE CASCADE,
    specialty_id  UUID NOT NULL REFERENCES specialties(id) ON DELETE CASCADE,
    is_primary    BOOLEAN DEFAULT FALSE,          -- выпускающая кафедра
    created_at    TIMESTAMP DEFAULT NOW(),
    UNIQUE (department_id, specialty_id)
);
CREATE INDEX idx_dept_spec_department ON department_specialties(department_id);
CREATE INDEX idx_dept_spec_specialty  ON department_specialties(specialty_id);


-- ---------- Учебный процесс ----------------------------------------------------
CREATE TABLE lecture_courses (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    specialty_id   UUID NOT NULL REFERENCES specialties(id) ON DELETE CASCADE,
    department_id  UUID REFERENCES departments(id) ON DELETE SET NULL,  -- РАСШИРЕНИЕ: кафедра, ведущая курс
    name           VARCHAR(500) NOT NULL,
    description    TEXT,
    semester       INT NOT NULL CHECK (semester BETWEEN 1 AND 12),
    total_hours    INT NOT NULL,
    lecture_hours  INT DEFAULT 0,
    practice_hours INT DEFAULT 0,
    lab_hours      INT DEFAULT 0,
    is_elective    BOOLEAN NOT NULL DEFAULT FALSE,                    -- РАСШИРЕНИЕ: курс по выбору
    tags           TEXT[] NOT NULL DEFAULT '{}',                      -- РАСШИРЕНИЕ: «спецдисциплина
```

```
        кафедры» и др.
    created_at     TIMESTAMP DEFAULT NOW(),
    updated_at     TIMESTAMP DEFAULT NOW(),
    UNIQUE (specialty_id, name, semester)
);
CREATE INDEX idx_courses_specialty  ON lecture_courses(specialty_id);
CREATE INDEX idx_courses_department ON lecture_courses(department_id);
CREATE INDEX idx_courses_tags       ON lecture_courses USING GIN (tags);


-- Занятие курса: лекция, практика или лабораторная
CREATE TABLE lectures (
    id               UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    course_id        UUID NOT NULL REFERENCES lecture_courses(id) ON DELETE CASCADE,
    title            VARCHAR(500) NOT NULL,
    annotation       TEXT,
    lecture_type     VARCHAR(20) NOT NULL CHECK (lecture_type IN ('lecture', 'practice', 'lab')),
    order_number     INT NOT NULL,
    duration_minutes  INT DEFAULT 90,
    tech_requirements TEXT,                        -- РАСШИРЕНИЕ: требования к техническим средствам (ЛР №2)
    tags             TEXT[] NOT NULL DEFAULT '{}', -- РАСШИРЕНИЕ: темы и тег спецдисциплины (ЛР №3)
    created_at       TIMESTAMP DEFAULT NOW(),
    updated_at       TIMESTAMP DEFAULT NOW(),
    UNIQUE (course_id, order_number)
);
CREATE INDEX idx_lectures_course ON lectures(course_id);
CREATE INDEX idx_lectures_type   ON lectures(lecture_type);
CREATE INDEX idx_lectures_tags   ON lectures USING GIN (tags);


-- Материалы занятия (нормализованно: одна строка — один материал). Дубль-агрегат — в MongoDB
CREATE TABLE lecture_materials (
    id           UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    lecture_id   UUID NOT NULL REFERENCES lectures(id) ON DELETE CASCADE,
    content_type VARCHAR(50) NOT NULL
                 CHECK (content_type IN ('text', 'presentation', 'pdf', 'video_link', 'code_sample',
                     'exercise')),
    title        VARCHAR(500),
    content_text TEXT,
    file_url     VARCHAR(1000),
    metadata     JSONB DEFAULT '{}',
    created_at   TIMESTAMP DEFAULT NOW(),
    updated_at   TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_materials_lecture ON lecture_materials(lecture_id);
CREATE INDEX idx_materials_type    ON lecture_materials(content_type);


-- ---------- Группы и студенты -----------------------------------------------------------
CREATE TABLE student_groups (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    specialty_id    UUID NOT NULL REFERENCES specialties(id) ON DELETE CASCADE,
    name            VARCHAR(50) NOT NULL UNIQUE,  -- «БСБО-01-23»
    enrollment_year INT NOT NULL,
    curator         VARCHAR(300),
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_groups_specialty ON student_groups(specialty_id);


CREATE TABLE students (
    id                  UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    group_id            UUID NOT NULL REFERENCES student_groups(id) ON DELETE CASCADE,
    first_name          VARCHAR(100) NOT NULL,
    last_name           VARCHAR(100) NOT NULL,
    patronymic          VARCHAR(100),
    email               VARCHAR(255) UNIQUE,
    phone               VARCHAR(20),
    student_card_number VARCHAR(20) UNIQUE,       -- номер студенческого билета / зачётной книжки
    enrollment_date     DATE NOT NULL,
    status              VARCHAR(20) DEFAULT 'active'
                        CHECK (status IN ('active', 'academic_leave', 'expelled', 'graduated')),
    created_at          TIMESTAMP DEFAULT NOW(),
    updated_at          TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_students_group  ON students(group_id);
CREATE INDEX idx_students_name   ON students(last_name, first_name);
CREATE INDEX idx_students_status ON students(status);
CREATE INDEX idx_students_fio_trgm ON students USING GIN ((last_name || ' ' || first_name) gin_trgm_ops);


-- РАСШИРЕНИЕ: выбор курсов по выбору (M:N студент <-> курс). Обязательные курсы группа
-- слушает целиком (по специальности и семестру), выборные — только записавшиеся студенты
CREATE TABLE student_electives (
    student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
    course_id  UUID NOT NULL REFERENCES lecture_courses(id) ON DELETE CASCADE,
    chosen_at  TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (student_id, course_id)
);
CREATE INDEX idx_electives_course ON student_electives(course_id);
```

```
-- ---------- Расписание и журнал посещений ----------------------------------------------
CREATE TABLE schedule (
    id              UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    lecture_id      UUID NOT NULL REFERENCES lectures(id) ON DELETE CASCADE,
    group_id        UUID NOT NULL REFERENCES student_groups(id) ON DELETE CASCADE,
    scheduled_date  DATE NOT NULL,
    week_start_date DATE NOT NULL,               -- понедельник недели (денормализовано: ключ партиций
        attendance)
    start_time      TIME NOT NULL,
    end_time        TIME NOT NULL,
    classroom       VARCHAR(50),
    teacher_name    VARCHAR(300),
    status          VARCHAR(20) DEFAULT 'scheduled' CHECK (status IN ('scheduled', 'completed', 'cancelled')),
    created_at      TIMESTAMP DEFAULT NOW(),
    updated_at      TIMESTAMP DEFAULT NOW(),
    CHECK (week_start_date = date_trunc('week', scheduled_date)::date)
);
CREATE INDEX idx_schedule_date       ON schedule(scheduled_date);
CREATE INDEX idx_schedule_week       ON schedule(week_start_date);
CREATE INDEX idx_schedule_group      ON schedule(group_id);
CREATE INDEX idx_schedule_lecture    ON schedule(lecture_id);
CREATE INDEX idx_schedule_group_date ON schedule(group_id, scheduled_date);


-- Журнал посещений — самая быстрорастущая таблица, секционирована по неделям (RANGE).
-- Внешние ключи не объявлены (как в методических указаниях): массовая вставка быстрее,
-- партиции свободно отсоединяются и переносятся; целостность обеспечивает приложение.
CREATE TABLE attendance (
    id              UUID NOT NULL DEFAULT uuid_generate_v4(),
    schedule_id     UUID NOT NULL,
    student_id      UUID NOT NULL,
    week_start_date DATE NOT NULL,               -- ключ партиционирования
    is_present      BOOLEAN NOT NULL DEFAULT FALSE,
    marked_at       TIMESTAMP DEFAULT NOW(),
    marked_by       VARCHAR(300),
    note            VARCHAR(500),
    created_at      TIMESTAMP DEFAULT NOW(),
    PRIMARY KEY (id, week_start_date)            -- PK обязан включать ключ партиционирования
) PARTITION BY RANGE (week_start_date);
CREATE UNIQUE INDEX uq_attendance_schedule_student ON attendance(schedule_id, student_id, week_start_date);
CREATE INDEX idx_attendance_student ON attendance(student_id);
CREATE INDEX idx_attendance_present ON attendance(is_present);


CREATE OR REPLACE FUNCTION create_weekly_partitions(start_date DATE, num_weeks INT) RETURNS VOID AS $$
DECLARE
    week_start DATE := date_trunc('week', start_date)::DATE;
    part TEXT;
BEGIN
    FOR i IN 0..num_weeks - 1 LOOP
        part := 'attendance_w' || to_char(week_start, 'YYYY_MM_DD');
        IF NOT EXISTS (SELECT 1 FROM pg_class WHERE relname = part) THEN
            EXECUTE format('CREATE TABLE %I PARTITION OF attendance FOR VALUES FROM (%L) TO (%L)',
                           part, week_start, week_start + 7);
        END IF;
        week_start := week_start + 7;
    END LOOP;
END;
$$ LANGUAGE plpgsql;


SELECT create_weekly_partitions('2026-02-09', 20);               -- весенний семестр 2026
CREATE TABLE attendance_default PARTITION OF attendance DEFAULT;   -- даты вне семестра


-- ---------- Представления ---------------------------------------------------------------
CREATE VIEW v_full_schedule AS
SELECT s.id AS schedule_id, s.scheduled_date, s.week_start_date, s.start_time, s.end_time, s.classroom,
       s.teacher_name, s.status AS schedule_status,
       l.id AS lecture_id, l.title AS lecture_title, l.lecture_type, l.tech_requirements,
       lc.id AS course_id, lc.name AS course_name, lc.semester,
       sg.id AS group_id, sg.name AS group_name,
       sp.name AS specialty_name, sp.code AS specialty_code,
       d.short_name AS department, i.short_name AS institute, u.short_name AS university
FROM schedule s
JOIN lectures l ON s.lecture_id = l.id
JOIN lecture_courses lc ON l.course_id = lc.id
JOIN specialties sp ON lc.specialty_id = sp.id
JOIN department_specialties ds ON sp.id = ds.specialty_id AND ds.is_primary
JOIN departments d ON ds.department_id = d.id
JOIN institutes i ON d.institute_id = i.id
JOIN universities u ON i.university_id = u.id
JOIN student_groups sg ON s.group_id = sg.id;


CREATE VIEW v_student_attendance_stats AS
SELECT st.id AS student_id, st.student_card_number, st.last_name || ' ' || st.first_name AS student_name,
       sg.name AS group_name,
       COUNT(a.id) AS total_classes,
```

```
       COUNT(*) FILTER (WHERE a.is_present) AS attended,
       COUNT(*) FILTER (WHERE NOT a.is_present) AS missed,
       ROUND(COUNT(*) FILTER (WHERE a.is_present)::NUMERIC / NULLIF(COUNT(a.id), 0) * 100, 1) AS
           attendance_percentage
FROM students st
JOIN student_groups sg ON st.group_id = sg.id
LEFT JOIN attendance a ON a.student_id = st.id
GROUP BY st.id, st.student_card_number, st.last_name, st.first_name, sg.name;


-- РАСШИРЕНИЕ: кто слушает курс — вся группа для обязательного курса, записавшиеся — для курса по выбору
CREATE VIEW v_course_listeners AS
SELECT lc.id AS course_id, lc.name AS course_name, lc.semester, st.id AS student_id, sg.name AS group_name
FROM lecture_courses lc
JOIN student_groups sg ON sg.specialty_id = lc.specialty_id
JOIN students st ON st.group_id = sg.id
WHERE NOT lc.is_elective
   OR EXISTS (SELECT 1 FROM student_electives e WHERE e.student_id = st.id AND e.course_id = lc.id);
```

Приложение Б. Маппинг индекса lectures (es_lectures_mapping.json)

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "analysis": {
      "analyzer": {
        "russian_custom": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": [
            "lowercase",
            "russian_stop",
            "russian_stemmer"
          ]
        }
      },
      "filter": {
        "russian_stop": {
          "type": "stop",
          "stopwords": "_russian_"
        },
        "russian_stemmer": {
          "type": "stemmer",
          "language": "russian"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "lecture_id": {
        "type": "keyword"
      },
      "course_id": {
        "type": "keyword"
      },
      "course_name": {
        "type": "text",
        "analyzer": "russian_custom",
        "fields": {
          "keyword": {
            "type": "keyword"
          }
        }
      },
      "title": {
        "type": "text",
        "analyzer": "russian_custom",
        "fields": {
          "keyword": {
            "type": "keyword"
          }
        }
      },
      "annotation": {
        "type": "text",
        "analyzer": "russian_custom"
      },
      "content_text": {
```

```
        "type": "text",
        "analyzer": "russian_custom"
      },
      "tech_requirements": {
        "type": "text",
        "analyzer": "russian_custom"
      },
      "lecture_type": {
        "type": "keyword"
      },
      "order_number": {
        "type": "integer"
      },
      "specialty_code": {
        "type": "keyword"
      },
      "department": {
        "type": "keyword"
      },
      "semester": {
        "type": "integer"
      },
      "tags": {
        "type": "keyword"
      },
      "updated_at": {
        "type": "date"
      }
    }
  }
}
```